



< Previous



Next >

Information as absence of redundancy

Bookmark this page

Problem: Does compression measure redundancy?

Redundancy obviously does not contribute to information. This is why compression is the best tool to measure information. For instance, unbalanced binary sequences that have more 0s than 1s, or more 1s than 0s, are highly redundant.

000000000100010000000000000001000001010000000000000000000000
01010000000000100000000000000000

They are expected to be compressible. Can we verify that?

The program `BinSeqGenerator.py` computes compression factors for **biased sequences**, using Zip (averaged over several trials; 100% compression corresponds here to the compression of a constant string).

For instance, if you execute: `BinSeqGenerator.py 1000 33` the program generates 1000-bit long sequences with 33% of 1s and tries to zip them.

Biased sequence 1

1/1 point (ungraded)

What average compression factor do you get for strings of length 1000 with a 22% bias?

☐ about 6% compression

☒ about 17% compression

☐ about 24% compression

☐ about 29% compression



Answer

Correct:

The compression is about 17% (as compared to the compression of a constant string).

Submit

You have used 1 of 1 attempt

Answers are displayed within the problem

Biased sequence 2

1/1 point (ungraded)

You may try several biases. What is the convexity of compression rate, as a function of bias?

☐ concave

☐ roughly linear

☒ convex



Answer

Correct:

Compression is strongly convex depending on bias. In other words, bias has to be strong for zip-compression to be efficient. This is due to the fact that one

be strong for zip compression to be efficient. This is due to the fact that one randomly occurring minority bit is sufficient to disrupt a long sequence of majority bits.

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem

Compressors such as Zip have their limitations. They can detect duplicate patterns, but they can't see all forms of redundancy. The best example of these limitations is offered by π . No common compressor is able to "see" the extreme redundancy hidden in π 's decimals. They are so redundant that you can get all of them once you know that they come from π .

Let's check whether Zip is blind to this redundancy.
Retrieve some of π 's decimals from the Web and save them to a file named `'Pi_decimals.txt'`.
Run the program `NumberCompression.py` in the presence of that file.
Read the output.

- Unsurprisingly, very round numbers, such as 10^{1000} , are highly compressible.
- Unsurprisingly, Zip is unable to compress a pseudo-random number.
- Surprisingly, the program seems able to compress `'Pi_decimals.txt'`! Why?

This small experiment reveals an error that any beginner in compressing techniques must encounter. The compressor takes text as input, and texts are coded in ASCII (8 bits per character). Coding π 's digits as typographical characters (ASCII) introduces redundancy that the compressor is able to detect and to eliminate. Numbers are indeed represented with codes between 48 and 57 (`\x30` and `\x39`).

Incompressibility

1/1 point (ungraded)

Go into `the program`. At the end, look at the "Compressing Pi" section. Convert π 's decimals to base 256 by adding a relevant line (compare with the random number section). Verify that π 's decimals aren't compressible after all. Copy the added line below.

TextSample = BaseChange(TextSample, Bytes)

✓

Answer: `.+BaseChange.+Bytes.+`

StrNum = BaseChange(Num, Bytes)

When the sequence is coded in base 256, all ASCII 8-bit codes are used and there is no longer any compression. π behaves like a random number (though it is not, of course).

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem



Legal

- [Terms of Service & Honor Code](#)
[Privacy Policy](#)
[Accessibility Policy](#)
[Trademark Policy](#)
[Sitemap](#)
[Cookie Policy](#)
[Your Privacy Choices](#)

Connect

- [Idea Hub](#)
[Contact Us](#)
[Help Center](#)
[Security](#)
[Media Kit](#)

